Solution: Problem Challenge 3

Problem Statement

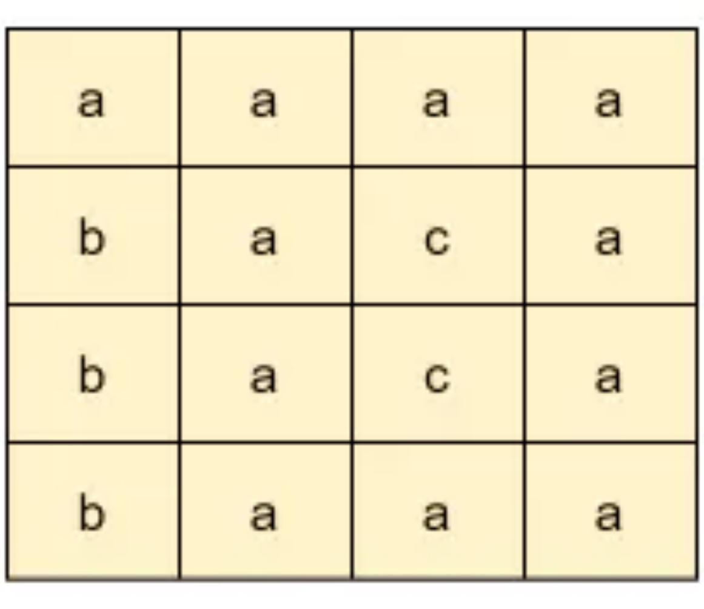
You are given a 2D matrix containing different characters, you need to find if there exists any cycle consisting of the same character in the matrix.

A **cycle** is a path in the matrix that starts and ends at the same cell and has **four or more cells.** From a given cell, you can move to one of the cells adjacent to it - in one of the four directions (up, down, left, or right), if it has the **same character value** of the current cell.

Write a function to find if the matrix has a cycle.

Example 1

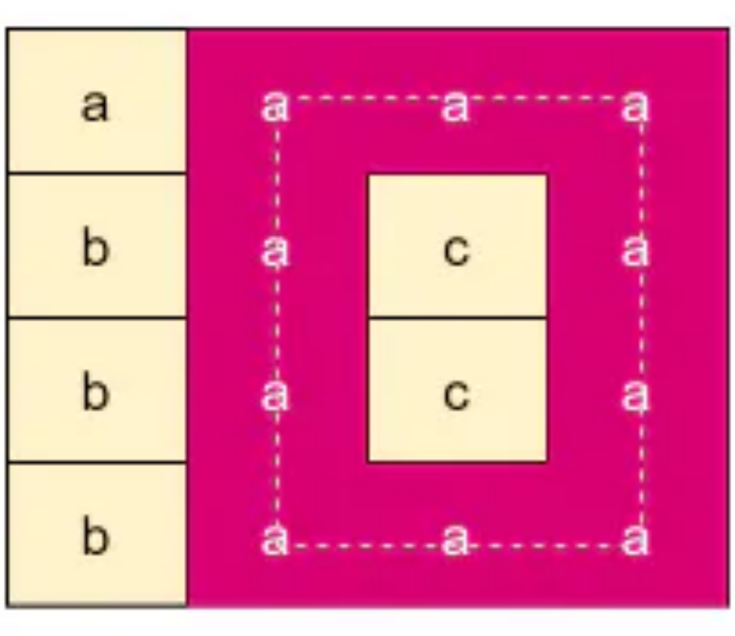
Input: matrix =



a	a	a	a
b	a	c	a
b	a	c	a
b	a	a	a

Output: true

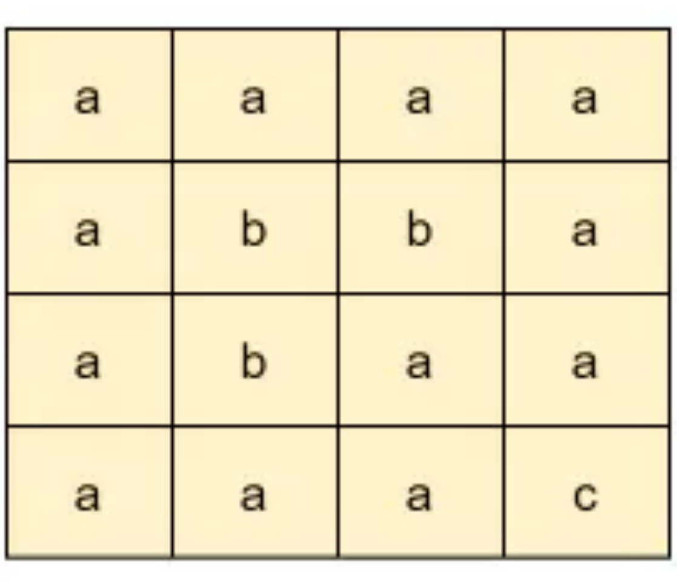
Explanation: The given matrix has a cycle as shown below:



a	a	a	a
b	a	c	a
b	a	c	a
b	a	a	a

Example 2

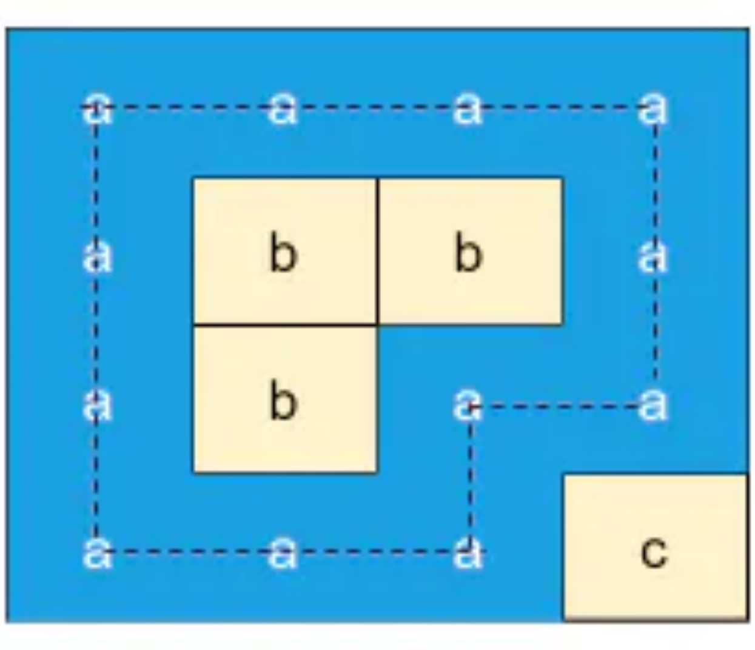
Input: matrix =



a	a	a	a
a	b	b	a
a	b	a	a
a	a	a	c

Output: true

Explanation: The given matrix has one cycle as shown below:



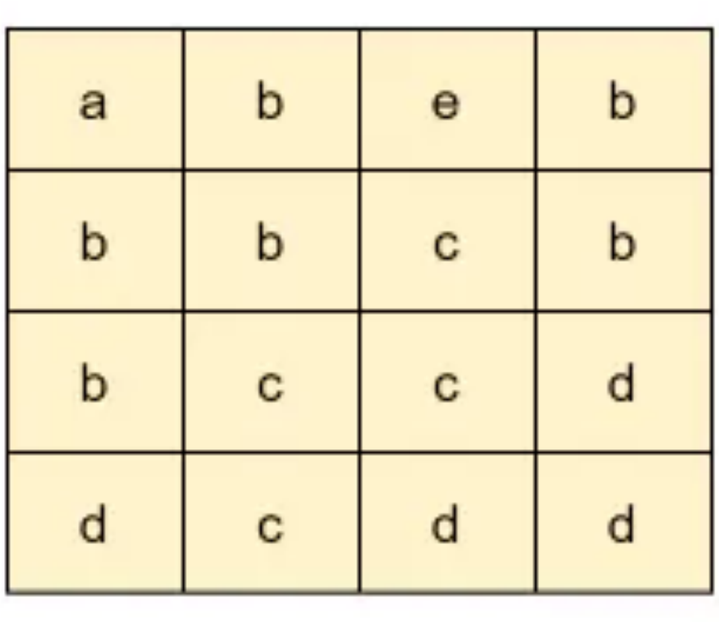
a	a	a	a
a	b	b	a
a	b	a	a
a	a	a	c

Example 3

Input: matrix =

Output: false

Explanation: The given matrix has no cycle.



a	b	e	b
b	b	c	b
b	c	c	d
d	c	d	d

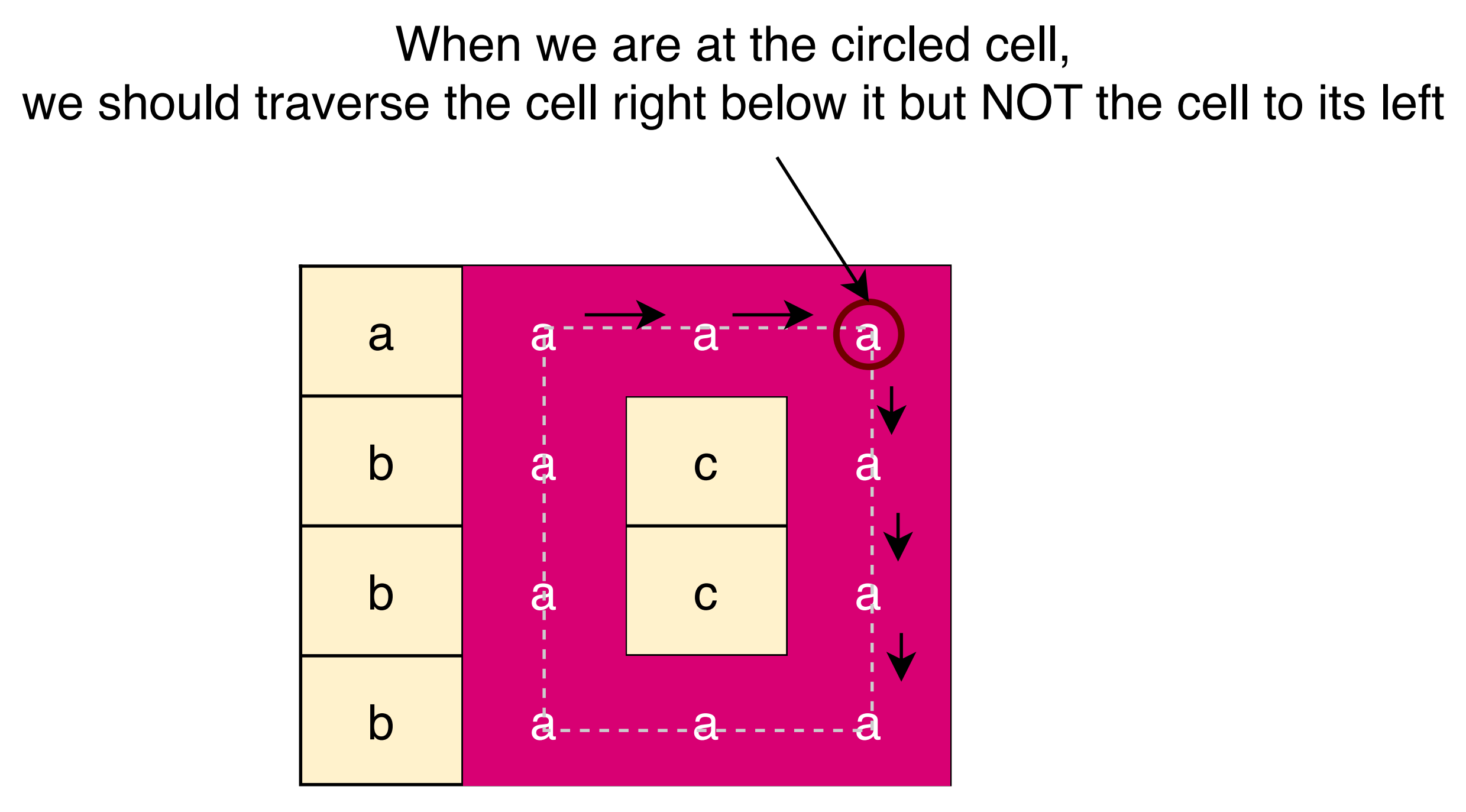
Solution

The question follows the Island pattern and is quite similar to Number of Islands problem.

We will traverse the matrix linearly to find any cycle. Each cycle is like an island having cells containing same values. Hence, we can use the **Depth First Search (DFS)** or **Breadth First Search (BFS)** to traverse a cycle, i.e., to find all of its connected cells with the same value.

Our approach for traversing the matrix will be similar to the one we used when searching for islands. We will keep another matrix to remember the cells that we have visited. From any given cell, we can perform DFS to traverse all the neighboring cells having the same character value.

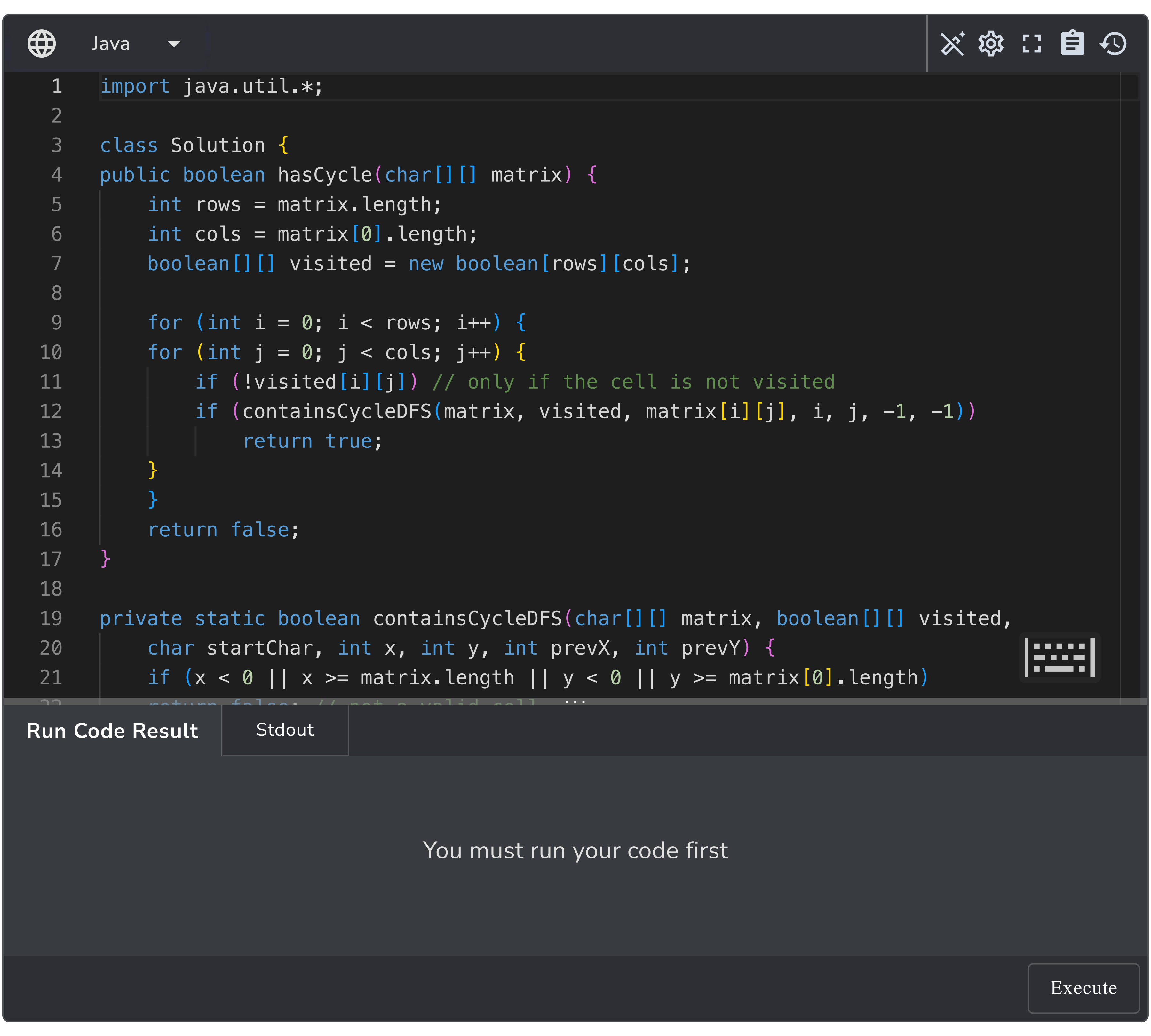
Whenever we reach a cell that have already been visited, we can conclude that we have found a cycle. This also means that we need to be careful to not start traversing the parent cell and wrongly finding a cycle. That is, while traversing, when initiating DFS recursive calls to all the neighboring cell, we should not start a DFS call to the pervious cell, **Here is the visual representation of the algorithm**:



When we are at the circled cell,
we should traverse the cell right below it but NOT the cell to its left

Code

Here is the code for this algorithm:



```java
import java.util.*;

class Solution {
public boolean hasCycle(char[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;
    boolean[][] visited = new boolean[rows][cols];

    for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (!visited[i][j]) // only if the cell is not visited
        if (containsCycleDFS(matrix, visited, matrix[i][j], i, j, -1, -1))
            return true;
    }
    }
    return false;
}

private static boolean containsCycleDFS(char[][] matrix, boolean[][] visited,
    char startChar, int x, int y, int prevX, int prevY) {
    if (x < 0 || x >= matrix.length || y < 0 || y >= matrix[0].length)
```

You must run your code first

Time Complexity

Time complexity of the above algorithm will be $O(M * N)$, where ‘M’ is the number of rows and 'N' is the number of columns of the input matrix. This is due to the fact that we have to traverse the whole matrix to find cycles.

Space Complexity

DFS recursion stack can go M*N deep when the whole matrix is filled with the same character. Hence, the space complexity will be $O(M * N)$, where ‘M’ is the number of rows and 'N' is the number of columns of the input matrix.

← Previous
Problem Challenge 3 (Medium)

Next →
Introduction To Two Heaps Pattern

☐ Mark as Completed